

Lab 9

Type classes in Haskell

Goals

In this lab you will learn to:

1. Understand the difference between types and kinds
2. Understand the basic type class hierarchy in Haskell
3. Implement type class instances
4. Define your own type classes

Resources

Table 9.1: Lab Resources

Resource	Link
Learn You a Haskell For Great Good!	http://learnyouahaskell.com/types-and-typeclasses#typeclasses-101
	http://learnyouahaskell.com/making-our-own-types-and-typeclasses#typeclasses-102
	http://learnyouahaskell.com/functors-applicative-functors-and-monoids#monoids
Ray Toal's Introduction to Haskell	https://cs.lmu.edu/~ray/notes/introhaskell/
Monoids without tears	https://fsharpforfunandprofit.com/posts/monoids-without-tears/
Haskell <code>Data.Eq</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-Eq.html
Haskell <code>Data.Ord</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-Ord.html
Haskell <code>GHC.Num</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/GHC-Num.html
Haskell <code>GHC.Enum</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/GHC-Enum.html
Haskell <code>GHC.Show</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/GHC-Show.html
Haskell <code>GHC.Read</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/GHC-Read.html
Haskell <code>Data.Monoid</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-Monoid.html
The Functional Programmer's Toolkit - Scott Wlaschin	https://youtu.be/Nrp_LZ-XGsY

9.1 Types and Kinds

Before starting to work with type classes, we need to briefly discuss 2 very important concepts: types and kinds.

We’ve seen that types are associated with values, bindings and function parameters, and that types can be *generic*, by having *type variables*.

Kinds are the next level in this chain: each type has a *kind* signature, that signals how many other types it takes to create the type. Types that aren’t generic like `Int` and `String` are represented by a single star (`*`) and types that are generic like the list type (`[]`) and `Maybe` are represented like a function signature: `* -> *`.

9.2 Writing generic code

One of the most important problems in programming is code reuse: what mechanism does the programming language offer to avoid duplicating logic?

A concrete example would be the implementation of a list data structure: we don’t want to implement a different list for each data type that we want the list to contain. Similarly imagine having to define a new tuple type for every possible combination of types.

One such mechanism that we’ve seen in Elm are *type variables*, which can be used to create generic data types and functions. Going back to the previous example, with type variables we can say that “this list can hold any data type” and “this list has the same logic for accessing the head and the tail of the list for any data type”.

We’ve also seen that in most cases the “any data type” part can be a bit too general: what if we want to express that “the logic for determining whether the list contains an element in the same, *for elements that have an equality relationship*”. Essentially we also want to be able to *constrain* the types that a type variable can be. In Elm we’ve seen that there are 3 possible constraints: `comparable`, `appendable` and `number`, without the possibility to define other constraints.

9.3 Introduction to type classes

Haskell uses *type classes* to allow the programmer to express these constraints. We’ve already seen some examples, like `Eq`, `Ord` and `Num`. The basic syntax for constraining a type variable is:

$$fnName :: (TClass_1\ var_1, \dots, TClass_n\ var_n) \Rightarrow Type_1 \rightarrow \dots \rightarrow Type_m \rightarrow RetType$$

Where $Type_i$ can be a normal type (e.g. `Int`, `String`), a type variable (e.g. `a`) or a type parametrized by one or more type variables (e.g. `[a]`, `Maybe a`, `(a, b)`).

So far it may seem that type variables and type classes are very similar to interfaces in Java: they provide a way to express that a set of actions that a class can perform, and then use the interface to say that “this function (method) works for any object that provides these methods”.

Without looking at the definition an implementation syntax just yet, type classes have one key advantage over interfaces: Since in Functional Programming we only have pure data declara-

Table 9.2: Standard type classes in Haskell

Name	Description	Minimal Complete Definition
<code>Eq</code>	Equality	<code>==</code> or <code>/=</code>
<code>Ord</code>	Ordering relationship	<code>compare</code> or <code><=</code>
<code>Bounded</code>	Types with a lower and upper bound	<code>minBound</code> and <code>maxBound</code>
<code>Enum</code>	Enumerable types	<code>toEnum</code> and <code>fromEnum</code>
<code>Show</code>	Printable types	<code>showsPrec</code> or <code>show</code>
<code>Read</code>	Types that can be parsed from a string	<code>readsPrec</code> or <code>readPrec</code>
<code>Semigroup</code>	Types that have an associative operation	<code><></code>
<code>Monoid</code>	Types that have an associative operation with an identity element	<code>mempty</code>
<code>Foldable</code>	Data structures that can be folded to a summary value	<code>foldMap</code> or <code>foldr</code>
<code>Traversable</code>	Data structures that can be traversed from left to right, performing an action on each element	<code>traverse</code> or <code>sequenceA</code>

9.5 Implementing instances

9.5.1 Implementing Show

Listing 9.5.1 of Instances.hs (Color, ShowColor)

Haskell code

```

6 data Color = Red | Green | Blue
18 instance Show Color where
19     show Red = "Red"
20     show Green = "Green"
21     show Blue = "Blue"

```

9.5.2 Implementing Eq

Listing 9.5.2 of Instances.hs (Color, EqColor)

Haskell code

```

6 data Color = Red | Green | Blue
10 instance Eq Color where
11     Red == Red = True
12     Green == Green = True
13     Blue == Blue = True
14     _ == _ = False

```

9.5.3 Implementing Ord

Listing 9.5.3 of Instances.hs (Medal, OrdMedal)

Haskell code

```
25 data Medal = Gold | Silver | Bronze deriving (Show, Eq)
29 instance Ord Medal where
30     compare Gold Gold = EQ
31     compare Gold _ = GT
32     compare Silver Silver = EQ
33     compare Silver Bronze = GT
34     compare Silver Gold = LT
35     compare Bronze Bronze = EQ
36     compare Bronze _ = LT
```

9.5.4 Implementing Enum

Listing 9.5.4 of Instances.hs (Medal, EnumMedal)

Haskell code

```
25 data Medal = Gold | Silver | Bronze deriving (Show, Eq)
40 instance Enum Medal where
41     toEnum 0 = Bronze
42     toEnum 1 = Silver
43     toEnum 2 = Gold
44
45     fromEnum Bronze = 0
46     fromEnum Silver = 1
47     fromEnum Gold = 2
```

Implementing Bounded

Listing 9.5.5 of Instances.hs (BoundedMedal)

Haskell code

```
51 instance Bounded Medal where
52     minBound = Bronze
53     maxBound = Gold
```

Enumerable types (Enum) and ranges

Listing 9.5.6 of Instances.hs (medals)

Haskell code

```
57 medals :: [Medal]
58 medals = [minBound..maxBound]
```

Table 9.3: How ranges are translated to functions defined in Enum

Range	Function
[from..]	enumFrom from
[from, next..]	enumFromThen from next
[from..to]	enumFromTo from to
[from, next..to]	enumFromThenTo from next to

9.5.5 Deriving instances

Some type classes have instance implementations that have a common pattern, and thus are repetitive to write by hand and don't constitute valuable information. For these cases, the compiler (GHC) can *automatically derive* (implement) the instances for us!

To ask for an automatic implementation, we simply write `deriving (Class1, Class2, ... ClassN)` after the data definition. The classes that can be derived include: `Eq`, `Ord`, `Show`, `Read`, `Enum`, `Bounded`.

9.6 Defining new type classes

The following example is taken from Chapter 8 of Learn You a Haskell for Great Good:

Lets define a `YesNo` type class that will be used to mimic the behavior of the conditional expressions in some dynamic languages (Python and JavaScript) where values can be *truthy* or *falsey*. For example in Python empty lists and the number 0 are falsey values, while non-empty lists and positive numbers are truthy values.

To define your type class we use the `class` keyword, followed by the name of the type class we intend to define and a type variable. The last one covers the types that represent the instance of the class in the functions defined on the class.

Listing 9.6.1 of YesNo.hs (`YesNo`)

Haskell code

```
4 class YesNo a where
5   yesno :: a -> Bool
```

Listing 9.6.2 of YesNo.hs (`YesNoList`, `YesNoInt`, `YesNoBool`)

Haskell code

```
9 instance YesNo [a] where
10   yesno [] = False
11   yesno _ = True
15 instance YesNo Int where
16   yesno 0 = False
17   yesno _ = True
21 instance YesNo Bool where
22   yesno b = b
```

We can also define type classes that are for *generic types*, like lists and `Maybe`:

Listing 9.6.3 of Container.hs (`Container`)

Haskell code

```
4 class Container c where
5   hasElem :: (Eq a) => c a -> a -> Bool
6   nrElems :: c a -> Int
```

Here `c` has kind `* -> *` because it needs a type (`a` in the `hasElem` and `nrElems` functions) to be a fully instantiated type.

Listing 9.6.4 of Container.hs (ContainerMaybe)

Haskell code

```

10 instance Container Maybe where
11   hasElem (Just x) e = x == e
12   hasElem _ _ = False
13
14   nrElems (Just _) = 1
15   nrElems _ = 0

```

Listing 9.6.5 of Container.hs (ContainerList)

Haskell code

```

19 instance Container [] where
20   hasElem l e = elem e l
21
22   nrElems l = length l

```

Notice that in both cases we only used the type name (i.e. `Maybe` and `[]`), **without specifying any type variables!** If we defined `Container` like `YesNo`, then we wouldn't be able to constrain the type variable `a` to be an instance of the `Eq` type class, which is needed for the `hasElem` function, by definition (after all, we can't search for elements if we can't compare them).

9.6.1 Specifying required type classes for a new type class definition

As mentioned above, the `Ord` type class requires that all types that are instances also implement `Eq`. We can specify this constraint in a manner similar to type constraints:

Listing 9.6.6: The Ord type class

Haskell code

```

class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<) :: a -> a -> Bool
  (<=) :: a -> a -> Bool
  (>) :: a -> a -> Bool
  (>=) :: a -> a -> Bool
  max :: a -> a -> a
  min :: a -> a -> a

```

9.6.2 Implementing instances for a subset of the parametrized type

Besides constraining the functions in the definition of a type class, we can also choose to implement a type class instance only for types that satisfy certain constraints (i.e. implement certain instances):

Listing 9.6.7 of YesNo.hs (YesNoLen)

Haskell code

```

26 class YesNoLen a where
27   yesnoLen :: a -> Int

```

Listing 9.6.8 of YesNo.hs (YesNoLenList)

Haskell code

```

31 instance (YesNo a) => YesNoLen [a] where
32   yesnoLen l = length $ filter yesno l

```

Here the `YesNoLen` type class is implemented only for instances of the list type that are `YesNo` types.

9.7 Case study: the Semigroup and Monoid type classes

Concept 9.7.1: Mathematical definition of a Semigroup

A semigroup is an algebraic structure consisting of a *set of elements* S and an *associative internal* operation $\oplus : S \times S \rightarrow S$. An operation is associative and internal if $\forall a, b, c \in S : (a \oplus b) \oplus c = a \oplus (b \oplus c)$.

Concept 9.7.2: Mathematical definition of a Monoid

A monoid is a semigroup, which besides the *associative internal* operation $\oplus$, also has an *identity element*, $1 \in S$. The neutral element should satisfy $1 \oplus a = a$ and $a \oplus 1 = a$ for any $a \in S$.

Concept 9.7.3: Definition of the Semigroup type class

A `Semigroup` is a type `a` which has an *associative internal* operation `<> :: a -> a -> a`. Associative means that `(a <> b) <> c == a <> (b <> c)`. In simpler terms you may think of semigroups as types that can be *appended*.

Concept 9.7.4: Definition of the Monoid type class

A `Monoid` is a semigroup, which besides the *associative internal* operation `<>`, also has an *identity element*, `mempty` defined on it. The neutral element should satisfy `mempty <> a = a` and `a <> mempty = a`. In simpler terms you may think of monoids as types that can be *appended* and have an instance that is *neutral* to the append operation.

9.7.1 The list monoid

Listing 9.7.1: Semigroup and Monoid instances for `[a]`

Haskell code

```
instance Semigroup [a] where
    (<>) = (++)

instance Monoid [a] where
    mempty = []
```

The String monoid

Since in Haskell `String` is just a type alias for `[Char]` we don't need to implement the `Semigroup` and `Monoid` instances again!

9.7.2 The Maybe monoid

Listing 9.7.2: Semigroup and Monoid instances for Maybe a

Haskell code

```
instance Semigroup a => Semigroup (Maybe a) where
  Nothing <> b      = b
  a      <> Nothing = a
  Just a  <> Just b  = Just (a <> b)

instance Semigroup a => Monoid (Maybe a) where
  mempty = Nothing
```

Note that we need to constrain the implementation, so that `a` is also a `Semigroup` for the case when both operands of `<>` are the `Just` variant.

Haskell REPL

```
> (Just [1]) <> (Just [2])
Just [1,2]
> (Just []) <> (Just [2])
Just [2]
> Nothing <> (Just [2])
Just [2]
```

9.7.3 The Sum and Product monoids

You've probably learned in high school about the semigroup and monoid concepts, applied to the set of natural numbers! There are actually two operations that can form a monoid with the set of natural numbers:

- Addition, with the identity element 0
- Multiplication, with the identity element 1

Now that we know that the `Int` and `Integer` types can be valid monoids, how should we implement the instance, given that we have multiple options?

We use the *newtype pattern*, where we define a new type (`Sum` and `Product`) to wrap the existing type (`Int`) and implement the instance for the wrapped types:

Listing 9.7.3: Semigroup and Monoid instances for Sum

Haskell code

```
newtype Sum a = Sum {getSum :: a}
instance Semigroup (Sum Int) where
  a <> b = Sum (getSum a + getSum b)

instance Monoid (Sum Int) where
  mempty = Sum 0
```

Listing 9.7.4: Semigroup and Monoid instances for Product

Haskell code

```
newtype Product a = Product {getProduct :: a}
instance Semigroup (Product Int) where
  a <> b = Product (getProduct a * getProduct b)

instance Monoid (Product Int) where
  mempty = Product 1
```

```
> nums = [1..4]
> getSum (foldl (<)> mempty (map Sum nums))
10
> getProduct (foldl (<)> mempty (map Product nums))
24
```

This is a great example of how we can *compose types* to change the behavior of functions, by implementing new type class instances.

9.8 Review questions

Question 9.8.1

*

What is the kind of the following types:

- `data Stuff = Stuff`
- `data Option a = None | Some a`
- `data Result err ok = Err err | Ok ok`
- `data Tree a = Nil | Node (Tree a) a (Tree a)`
- `data Map k v = Map [(k, v)]`

Question 9.8.2

*

Does Elm have semigroups or monoids? If yes what are they called?

Question 9.8.3

**

Of the following operations, which are semigroups and which are monoids? In the case of monoids, what is the neutral element for the operation? In the case when the operation is not a semigroup, which requirement is not satisfied?

- The `&&` (and) function
- The `||` (or) function
- The `xor` function
- The `<` function
- The `min` function for `Int`
- The `min` function for `Integer`
- The `max` function for `Int`
- The `max` function for `Integer`
- The `\` function for `Int`
- The `\` function for `Float`

Hints:

Re-read the definitions of semigroups and monoids, paying close attention to the requirements for each type class.

9.9 Practice problems

Exercise 9.9.1

*

Given the following data definition: `data Op = Add | Sub | Mul | Div`, implement `Show` such that the corresponding mathematical operator is shown for each instance (i.e. `+` for `Add`, `-` for `Sub`, etc).

Exercise 9.9.2

*

Implement the `YesNo` type class for the `Maybe` type.

Exercise 9.9.3

**

Implement the `Container` type class for a binary tree data type, which is defined as: `data Tree a = Nil | Node (Tree a) a (Tree a)`.

Haskell REPL

```
> t = Node (Node Nil 1 Nil) 2 (Node Nil 3 Nil)
> hasElem t 2
True
> hasElem t 3
True
> hasElem t 4
False
```

Exercise 9.9.4

**

Read about the `All` and `Any` monoids in the “Monoids” section of Chapter 11 - “Functors, Applicative Functors and Monoids” of Learn You a Haskell for Great Good² and use them to implement the `allUpper` function, which checks if the string contains only uppercase characters in two ways:

- Using the `All` monoid
- Using the `Any` monoid

Hints:

You might want to use (some of) the following functions: `foldl`.

<http://learnyouahaskell.com/functors-applicative-functors-and-monoids#monoids>

Exercise 9.9.5

Given the newtype `Rev`, defined as `newtype Rev a = Rev a deriving (Eq, Show)` implement `Ord` such that the `sortRev :: (Ord a) => [a] -> [a]` function, defined as `sortRev = sortOn Rev` will sort the list in reverse.

```
> import qualified Data.List as L
> l = [4, 2, 5, 1, 6]
> L.sort l
[1, 2, 4, 5, 6]
> sortRev l
[6, 5, 4, 2, 1]
```

Exercise 9.9.6

Read about the `Foldable` type class in the “Using monoids to fold data structures” subsection of Chapter 11 - “Functors, Applicative Functors and Monoids” of Learn You a Haskell for Great Good and reimplement the `allUpper` function using the `foldMap` function and one of the `All` or `Any` monoids.

Hints:

You’ll have to import the `Data.Foldable`, `Data.Monoid` and `Data.Char` modules. The resulting code should be shorter than the previous implementation.

Given the type class `Graph` and the newtypes `TupleGraph` and `NeighborListGraph`, implement the `neighbors` function that returns the neighbors of a given node for each graph type. If the node is not in the graph, the empty list should be returned.

Subpoint 9.9.7.1

Implement the `bf` function, that performs breadth-first search on a directed acyclic graph.

Subpoint 9.9.7.2

Implement the `df` function, that performs depth-first search on a directed acyclic graph.

Both functions should return the list of visited nodes in the order corresponding to the used algorithm.

Hints:

Questions you should think about before writing the code:

1. What data structures does each algorithm (bfs and dfs) use?
2. How can you implement each of these data structures in Haskell using lists?

As a starting point, both algorithms use 2 data structures:

- One to keep track of the nodes that will be visited
- One to keep track of the nodes that were already visited

Based on this observation, a generic helper function could have the signature:

`helper :: (Graph g, Eq a) => [a] -> g -> [a] -> [a]` where

- The first parameter represents the data structure that is used to keep track of the nodes that will be visited
- The second parameter is the graph
- The third parameter is the data structure used to keep track of the nodes that were already visited
- The return value represents the nodes to be visited in the next iteration